

AgriHGT: Heterogeneous Graph Transformer for Multi-Crop Leaf Disease Classification

Author One, Author Two, Author Three

Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh

{author1, author2, author3}@example.com

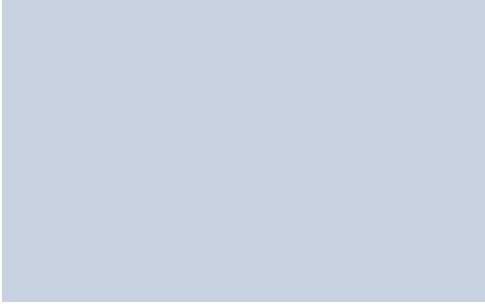


Fig. 1. Overview of the AgriHGT methodology.

Abstract—AgriHGT is a two-stage visual-relational framework for 28-class leaf disease classification on the Agri-Vision4 dataset (Tomato, Papaya, Zucchini, Bottle Gourd). Partially fine-tuned visual backbones provide one feature vector per leaf image, and a Heterogeneous Graph Transformer propagates information among leaf-image, crop, disease, and disease-category nodes before a gated residual fusion and a leaf-disease compatibility decoder. A duplicate-aware, leakage-controlled split, a label-shortcut-free message-passing partition, seven backbones, three message-passing depths, and a nine-configuration component ablation are evaluated. [Results to be inserted from the generated tables.]

Index Terms—Plant disease classification, heterogeneous graph transformer, graph neural network, self-supervised learning, DINOv3, Agri-Vision4.

I. INTRODUCTION

II. RELATED WORK

III. MATERIALS AND METHODS

A. Overview

AgriHGT is a two-stage visual-relational classifier (Fig. 1). First, a pretrained visual backbone is partially fine-tuned on the training partition and used to extract one fixed feature vector per retained leaf image. Second, these features become `leaf_image` nodes in a heterogeneous graph with crop, disease, and disease-category nodes, on which relation-aware message passing is performed with the Heterogeneous Graph Transformer (HGT) [?]. The graph representation is fused with a residual visual representation before disease prediction.

B. Dataset

Agri-Vision4 [?] contains leaf images of Tomato, Papaya, Zucchini, and Bottle Gourd in 28 crop-specific healthy and disease classes, collected in Bangladeshi fields under natural light and validated by agricultural experts. It provides original

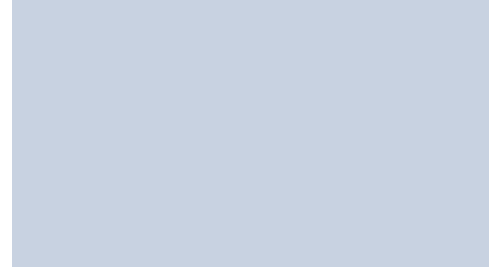


Fig. 2. Original and augmented images across the four crops of Agri-Vision4.

images and augmented copies (rotation, flipping, brightness and contrast adjustment, noise); augmented images form a large share of every crop (Fig. 2). Before duplicate removal there were 28,121 files, of which 5,246 were originals (Table I). Because class frequencies are unequal, accuracy is reported with macro-F1 and other class-balanced metrics.

C. Leakage-Controlled Data Partitioning

Exact duplicates. An MD5 hash was computed for all 28,121 files, and one file per identical-hash group was kept. This removed 1,255 byte-identical duplicates, leaving 26,866 images (4,304 original, 22,562 augmented).

Duplicate-aware initial split. A random split over originals and augmentations could put an image and its transformed copy in different partitions and inflate validation or test scores, so the originals were split first with class-stratified 70/15/15 sampling (3,004/650/650). Perceptual hashes with `hash_size=16` (256 bits) were then computed, and each augmented image was compared with originals of the same crop and class; if the minimum Hamming distance was at most six, it was linked to the closest original and assigned to its partition. This linked 1,552 augmented images; the other 21,010 could not be confidently associated and were assigned to training.

Near-duplicate audit. Same-class perceptual hashes were then compared across partitions for original–original, original–augmented, and augmented–augmented pairs. Whenever a cross-partition pair had Hamming distance ≤ 6 , the validation or test image was moved to training, repeating until no violation remained; successive rounds found 41, 11, 3, and 0 violating pairs.

Per-class support control. Because the previous steps could remove the only validation or test image of a minority class, a class lacking either was given an eligible training

TABLE I
ORIGINAL IMAGES PER CROP-SPECIFIC CLASS BEFORE DUPLICATE REMOVAL (TOTAL 5,246)

Bottle Gourd (1,658)		Papaya (1,684)		Tomato (916)		Zucchini (988)	
Alternaria Leaf Blight	303	Bacterial Blight	183	Downy	57	Angular Leaf Spot	120
Anthrachnose	276	Carica Insect Hole	318	Healthy	288	Anthrachnose	129
Downy Mildew	286	Curl'd Yellow Spot	538	Mosaic	195	Downy Mildew	153
Early Alternaria L. Blight	179	Healthy Leaf	189	Spot	311	Dry Leaf	67
Fungal Damage Leaf	39	Mosaic Virus	119	White Spot	65	Healthy	88
Healthy	260	Pathogen Symptoms	286			Insect Damage	78
Mosaic Virus	315	Yellow Necrotic Spot Holes	51			Iron Chlorosis Damage	65
						Xanthomonas Leaf Spot	86
						Yellow Mosaic Virus	202

image, after which the audit was repeated; if the reassigned image created a new near-duplicate relation, it was returned and another candidate tried. The final split has 25,122 training, 896 validation, and 848 test images, with at least one validation and one test image per class, and is shared by all backbone experiments. It is *leakage-controlled* rather than leakage-free: the audit covers same-class pairs and the chosen hash and threshold, so the supported claim is that no same-class cross-partition pair exists at Hamming distance ≤ 6 .

D. Visual Backbones

Each image’s backbone feature initializes its `leaf_image` node and defines the `similar_to` relations, so the graph changes with the backbone. Seven backbones spanning convolutional, transformer, supervised, and self-supervised families used in recent plant-disease studies [?], [?], [?] were compared (Table II), both to find the strongest one and to test whether heterogeneous graph learning behaves consistently across representations. ResNet50 is a standard residual CNN baseline whose residual connections ease deep training, used widely in plant-disease transfer learning [?], [?]. DenseNet121 connects each layer to all earlier outputs, encouraging feature reuse so that small lesions and larger texture patterns both remain available. EfficientNet-B3 scales depth, width, and resolution jointly, suiting symptoms at multiple scales, and performs strongly on leaf disease [?]. ConvNeXt-Tiny is a modern CNN with transformer-inspired design, a middle point showing whether gains need attention or also arise in a modernized CNN. Swin-Tiny is a supervised hierarchical transformer with shifted-window local attention, applied to plant disease in [?], [?], [?]. DINOv2 ViT-B/14 is self-supervised, learning from unlabeled images an embedding space that may preserve visual structure beyond supervised classes, which matters because the graph needs meaningful neighborhoods, not only a strong classifier [?], [?]. DINOv3 ViT-B/16, a newer DINO model, tests whether a newer self-supervised representation helps; the DINOv2–DINOv3 comparison isolates representation generation within one family.

E. Backbone Adaptation and Feature Extraction

Generic pretrained features may emphasize shape, background, and global appearance rather than subtle color and texture cues, so each backbone was first adapted on the training partition with a temporary 28-class head, fine-tuning only the last stage: `layer4` for ResNet50; `denseblock4`,

TABLE II
VISUAL BACKBONES AND THEIR ROLE IN AGRIHGT

Backbone	Architecture	Pretrain.	Reason for inclusion
ResNet50	Residual CNN	Sup.	Standard CNN reference
DenseNet121	Dense CNN	Sup.	Feature reuse
EfficientNet-B3	Scaled CNN	Sup.	Compound scaling
ConvNeXt-Tiny	Modern CNN	Sup.	CNN–transformer bridge
Swin-Tiny	Hierarchical ViT	Sup.	Local-window attention
DINOv2 ViT-B/14	ViT	Self-sup.	Strong SSL features
DINOv3 ViT-B/16	ViT	Self-sup.	Recent DINO generation

`norm5`, and classifier for DenseNet121; `features[7:]` for EfficientNet-B3; `features[7]` for ConvNeXt-Tiny; the final stage and terminal norm for Swin-Tiny; and the final two blocks and terminal norms for DINOv2 and DINOv3, each with its head. Partial fine-tuning limits cost and preserves low- and mid-level pretrained features on a dataset far smaller than the pretraining data. Validation accuracy was monitored every epoch (with patience-based early stopping where used), the best checkpoint was restored, the head was replaced by an identity, and the frozen backbone ran one deterministic pass without augmentation over all 26,866 images at its native resolution and normalization. This gives feature matrices of $26,866 \times d$ with $d = 2,048$ (ResNet50), 1,024 (DenseNet121), 1,536 (EfficientNet-B3), and 768 (ConvNeXt-Tiny, Swin-Tiny, DINOv2, DINOv3). Each dimension was standardized as $x' = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$ with statistics from the training partition only, applied unchanged to validation and test, separately per backbone.

F. Heterogeneous Graph Construction

Graph learning has captured relations among image regions or leaves [?], [?], and agricultural knowledge graphs relate crops, diseases, pests, and symptoms [?], [?], [?]; AgriHGT combines both (Fig. 3). One graph per backbone has 26,905 nodes: 26,866 `leaf_image`, 4 `crop`, 28 `disease`, and 7 `disease_category` nodes (Healthy, Viral, Bacterial, Fungal, Pest, Nutrient Deficiency, Other), with categories fixed from class names before training [?], [?]. Leaf nodes take the standardized backbone vector; non-image nodes have no pixels and receive learnable input embeddings.

Six forward relations are used: `grown_on` (`leaf`→`crop`), `affects` (`crop`→`disease`), `is_type_of` (`disease`→`category`), `similar_to` (`leaf`→`similar leaf`), `has_disease` (selected training `leaf`→`disease`), and `co_category` (`disease`→`disease of the same category`).

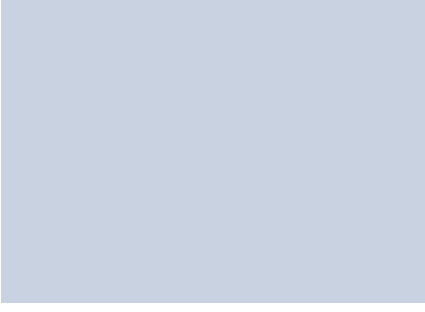


Fig. 3. AgriHGT heterogeneous graph: leaf_image, crop, disease, and disease_category nodes and their typed relations.

Adding a reverse for each and a self-loop per node gives 13 relation types. For `similar_to`, following feature-space similarity graphs [?], each image links to its five nearest neighbors of the same crop under

$$d_{\cos}(\mathbf{x}_i, \mathbf{x}_j) = 1 - \frac{\mathbf{x}_i^T \mathbf{x}_j}{\|\mathbf{x}_i\|_2 \|\mathbf{x}_j\|_2}, \quad (1)$$

restricting to the crop to reduce the influence of crop-level leaf shape, giving 134,330 forward edges, recomputed per backbone.

Message and supervision partition. Linking every supervised training image to its true disease would leak its label along `leaf_image`→`disease`→`leaf_image`. The training partition was therefore split, class-stratified 75/25 with originals and linked derivatives kept together, into 18,881 `train_msg` images, which keep `has_disease` edges but are excluded from the loss, and 6,241 `train_sup` images, which have no `has_disease` edge and alone form the loss. Validation and test nodes also lack `has_disease` edges, so `train_sup`, validation, and test nodes follow the same rule. With reverse relations and self-loops the graph has 387,301 edges.

G. AgriHGT Architecture

AgriHGT has three components: heterogeneous message passing, gated fusion of graph and visual representations, and a leaf-disease compatibility decoder (Fig. 4).

Projection and HGT. Node features of type $\tau(v)$ are mapped to 256 dimensions, $\mathbf{h}_v^{(0)} = \tanh(W_{\tau(v)}\mathbf{x}_v + \mathbf{b}_{\tau(v)})$. HGT uses type-dependent transformations and relation-aware attention [?], fitting a graph in which a leaf-similarity message means something different from a crop-disease or taxonomy message. We use 4 heads, hidden size 256, dropout 0.2, and depths $n_{\text{layers}} \in \{0, 1, 2\}$. The zero-layer model keeps the projections, learnable non-image embeddings, residual branch, fusion, and decoder but no message passing; since the original pyHGT code still built one convolution at depth 0, it was modified to perform none.

Gated residual fusion. The standardized image feature is projected as $\mathbf{r}_i = \text{LayerNorm}(W_r\mathbf{x}_i + \mathbf{b}_r)$. With graph-updated leaf representation $\mathbf{h}_i$,

$$\mathbf{g}_i = \sigma(W_g[\mathbf{r}_i \parallel \mathbf{h}_i] + \mathbf{b}_g), \quad \mathbf{z}_i = \mathbf{g}_i \odot \mathbf{r}_i + (1 - \mathbf{g}_i) \odot \mathbf{h}_i, \quad (2)$$

where $\parallel$ is concatenation. The gate bias starts at 2.0 ($\sigma(2) \approx 0.88$), so the model first favors the visual path and learns how much graph information to admit.

Compatibility decoder. Leaf and disease features are projected to 128 dimensions, $\mathbf{u}_i = W_l\mathbf{z}_i + \mathbf{b}_l$ and $\mathbf{v}_j = W_d\mathbf{d}_j$, and scored against all 28 disease nodes:

$$s_{ij} = W_o \text{Dropout}_{0.1}[\text{ReLU}(\mathbf{u}_i + \mathbf{v}_j)] + b_o, \quad (3)$$

and the prediction is $\hat{y}_i = \arg \max_j s_{ij}$, the disease whose representation best matches the leaf.

H. Training and Transductive Setting

Training is full-batch over the whole graph, with loss only on the 6,241 `train_sup` nodes. Class weights $\tilde{w}_c = 1/n_c$ over `train_sup` counts are normalized to mean one, and

$$\mathcal{L} = -\frac{1}{|S|} \sum_{i \in S} w_{y_i} \log \frac{\exp(s_{i, y_i})}{\sum_{j=1}^{28} \exp(s_{ij})}. \quad (4)$$

AdamW (learning rate 10^{-3} , weight decay 10^{-4}), gradient clipping at 1.0, and cosine decay toward 10^{-6} are used. ResNet50, Swin-Tiny, and DenseNet121 runs use up to 300 epochs with patience 300 (the full budget, keeping the best validation checkpoint); ConvNeXt-Tiny, EfficientNet-B3, DINOv2, and DINOv3 use up to 400 epochs with patience 50; the minimum improvement is 10^{-4} . Depths 0, 1, and 2 share one partition per backbone, and the preferred depth and checkpoint are chosen by validation accuracy only; test results never drive selection. The setting is *transductive*: validation and test leaf nodes are in the graph and take part in message passing, but their labels are excluded from the loss and they have no `has_disease` edges. Results therefore measure classification of unlabeled nodes already in the graph, not fully inductive prediction for images added after training.

I. Reference Models and Ablation Design

Logistic-regression reference. A multinomial logistic regression (up to 2,000 iterations) is trained per backbone on the same standardized features without any graph, crop, disease, or category nodes, HGT, fusion, or decoder, reflecting conventional feature-only recognition [?], [?]. It uses all 25,122 training labels, since no label-bearing edges require a message/supervision split, so it is a visual-feature reference rather than a matched supervision ablation. **Zero-layer reference.** HGT-0 keeps the rest of AgriHGT, so comparing it with HGT-1 and HGT-2 isolates message propagation.

Component ablation. With DINOv3 features, two HGT layers (best DINOv3 validation depth) form the full reference A0, and one component is changed at a time (Table III), keeping hidden size 256, 4 heads, dropout 0.2, the MLP decoder, the weighted loss, the split, the feature bank, the `train_msg/train_sup` partition, and the validation and test sets. For A2–A5 the graph is rebuilt without only the named nodes or relations (the within-crop neighbor graph is kept for A3–A5 and removed only in A2). A6 keeps the structure but zeroes the meta-node embeddings, A7 feeds $\mathbf{h}_i$ directly to the decoder, and A8 replaces Eq. (2) with $\mathbf{z}_i = 0.5\mathbf{r}_i + 0.5\mathbf{h}_i$. A0 and A2–A8 use up to 400 epochs,

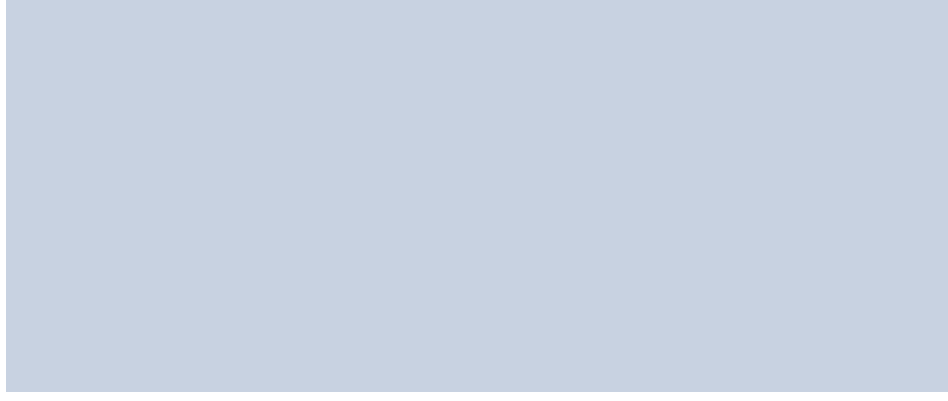


Fig. 4. AgriHGT architecture: visual feature extraction, heterogeneous graph construction, relation-aware HGT message passing, residual visual fusion, and disease prediction.

TABLE III
COMPONENT ABLATION CONFIGURATIONS (DINOv3 FEATURES)

ID	Modification relative to full AgriHGT (A0, two HGT layers)
A0	None: all relations and components retained
A1	Zero HGT layers: no GeneralConv message passing
A2	Remove similar_to and its reverse
A3	Remove crop nodes, grown_on, affects, and reverses
A4	Remove category nodes, is_type_of, co_category, and reverses
A5	Remove has_disease and reverse (no visual grounding of diseases)
A6	Replace learnable crop/disease/category embeddings with zeros
A7	Remove the residual visual bypass (graph representation only)
A8	Replace the learned gate with a fixed equal-weight average

AdamW (10^{-3} , 10^{-4}), cosine decay to 10^{-6} , patience 50, minimum improvement 10^{-4} , and seed 42; A1 reuses the zero-layer DINOv3 run under the same settings. With a single seed, differences are component-level observations, which the paired McNemar tests below qualify.

J. Evaluation Metrics

For N test images with per-class counts TP_c , FP_c , FN_c , TN_c , we report accuracy, precision $P_c = TP_c / (TP_c + FP_c)$, recall $R_c = TP_c / (TP_c + FN_c)$, specificity $Sp_c = TN_c / (TN_c + FP_c)$, $F1_c = 2P_cR_c / (P_c + R_c)$, and Jaccard $J_c = TP_c / (TP_c + FP_c + FN_c)$, with macro averages $\frac{1}{28} \sum_c (\cdot)$ that weight every class equally; balanced accuracy is the macro recall. Chance-corrected agreement is measured by Cohen’s $\kappa = (p_o - p_e) / (1 - p_e)$, with $p_e = \sum_c n_c \hat{n}_c / N^2$, and by the multiclass Matthews correlation

$$MCC = \frac{C N - \sum_c \hat{n}_c n_c}{\sqrt{(N^2 - \sum_c \hat{n}_c^2)(N^2 - \sum_c n_c^2)}}, \quad (5)$$

where C is the number of correct predictions and n_c , $\hat{n}_c$ are true and predicted class counts. From predicted probabilities we report top-3 accuracy, macro one-vs-rest ROC-AUC, negative log-likelihood (NLL), and expected calibration error $ECE = \sum_b \frac{|B_b|}{N} |\text{acc}(B_b) - \text{conf}(B_b)|$ over 15 confidence bins. Accuracy has a 95% Wilson interval

$$\left(\hat{p} + \frac{z^2}{2N} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}} \right) / \left(1 + \frac{z^2}{N} \right), \quad z = 1.96, \quad (6)$$

macro-F1 a 95% bootstrap interval (2,000 resamples of the test set), and each ablation is compared with A0 on the same 848 images by an exact McNemar test on the discordant pairs (n_{01}, n_{10}) , $p = 2 \sum_{k \leq \min(n_{01}, n_{10})} \binom{n_{01} + n_{10}}{k} 2^{-(n_{01} + n_{10})}$. All values are computed by a released script directly from saved predictions.

K. Experimental Setup

Experiments ran on Kaggle with one NVIDIA Tesla T4 GPU. PyTorch and Torchvision served backbone training and feature extraction; DINOv2 was loaded from its official implementation and DINOv3 ViT-B/16 via `timm`; pyHGT provided the graph and HGT layers (with the zero-layer change of Section III-G); PyTorch Geometric supported graph operations; scikit-learn provided nearest-neighbor search, logistic regression, and evaluation; Pandas and NumPy handled data; and Pillow and ImageHash handled images and perceptual hashing. The split was fixed across backbones, seed 42 was used for the main and ablation experiments, and a fixed hold-out validation set (not k -fold cross-validation) selected checkpoints and depths. All experiments are single-seed; multi-seed variability was not estimated and is a limitation.

IV. RESULTS

V. DISCUSSION AND LIMITATIONS

VI. CONCLUSION

